

How Well Do Cheap Phishing Detectors Generalize? A Leakage-Aware, Cross-Corpus and Cost-Aware Benchmark of Small Open LLMs

Jahid Ibna Sinha
ID 011221376
United International University
Dhaka, Bangladesh

Md. Jakaria Alam Saimon
ID 011221002
United International University
Dhaka, Bangladesh

Nowshin Anjum Faria
ID 011221372
United International University
Dhaka, Bangladesh

Abstract—Published phishing email detectors report 97 to 99% accuracy, almost always measured inside a single corpus. We ask what they are worth on mail from a corpus they have never seen, once the emails that public corpora share with each other are removed. Comparing nine public sources, we find that 87% of SpamAssassin, 91% of Ling-Spam and 36% of Enron have a near duplicate inside the widely used Kaggle phishing set, while an exact-match check finds 2 of the SpamAssassin copies. Removing the duplicates lowers a transfer score from 0.966 to 0.672 F1, and removing the same number of training emails at random instead changes nothing, which separates leakage from lost data. An LLM annotator finds that only 15 to 25% of the positive emails in these corpora are phishing rather than bulk spam. On decontaminated leave-one-corpus-out data we compare classical models, a fine-tuned DistilBERT, six small open LLMs, three published phishing detectors and two commercial models on identical emails, with false alarm rates and the measured dollar cost. Zero-shot Qwen-2.5-7B reaches 0.928 mean F1 with 3% false alarms at three US cents per 1,000 emails, Gemini-3.1-Flash-Lite reaches 0.960, and a two-stage detector that sends only flagged mail to an LLM reaches 0.911 at 1.1% false alarms for 0.0161 dollars per 1,000. We also show that the AI-written phishing corpus used here carries a give-away token which a naive comparison blames for 16 points of recall, but which a paired test clears (0.9 points), and that few-shot examples taken from old corpora, rather than few-shot prompting itself, are what lower performance on AI-written mail. The whole study ran on one laptop for 1.58 US dollars.

Index Terms—phishing detection, large language models, dataset leakage, near-duplicate detection, cross-corpus evaluation, cost

I. Introduction

Phishing email is still one of the most common ways into an organization. The attacker does not break a technical control. They write a message that looks like it came from a bank, a colleague or a supplier, and the reader hands over the password. Generative AI made writing such mail cheap: Heiding et al. [1] showed that phishing written by GPT-4 already gets click rates close to phishing written by human experts.

Detection results in the literature look excellent. ChatSpamDetector reports 99.70% accuracy with GPT-4 [2], a fine-tuned RoBERTa reports 98.45% [3], and a five agent LLM system reports 97.89% [7]. When we

reproduced the setting with TF-IDF and logistic regression we also got 97 to 99%. A method older than deep learning matching GPT-4 is a warning sign: it suggests the benchmark, not the model, is doing the work.

This paper asks what these detectors are worth when the test mail comes from a corpus the model has never seen, and when the emails that the corpora share with each other are removed first. Three things make the question concrete. First, the popular corpora are not independent. Aggregated sets such as the Kaggle phishing email set are built from older collections, so the same email can sit on both sides of a train and test split even when the two sides come from datasets with different names. Second, the cost of running a detector is never reported, although for a university or a small company it decides what can be deployed at all. Third, a single F1 score hides the thing an operator cares about most, which is how much legitimate mail gets flagged.

Our contributions are:

- 1) A measurement of email overlap between nine public phishing and spam sources at three levels of strictness, with every near-duplicate pair verified by exact Jaccard similarity and the matcher itself validated against brute force. Exact matching, the check most people would run, finds almost none of the overlap.
- 2) A decontaminated leave-one-corpus-out benchmark, with a control that removes the same number of training emails at random. The control separates the effect of leaked emails from the effect of simply having less data, which no earlier work on this question reports.
- 3) A measurement of what the corpora actually contain. An LLM annotator with a second annotator for agreement finds that only 15 to 25% of the positive emails in these corpora are phishing; the rest is bulk spam.
- 4) A comparison on identical emails of classical models, a fine-tuned DistilBERT, six small open LLMs, three published phishing detectors and two current commercial models, reporting the false alarm rate and the measured dollar cost next to accuracy.

- 5) An analysis of AI-written phishing that finds a giveaway token in the E-PhishLLM corpus and then shows, with a paired test on sanitised copies of the same emails, that it is not what the detectors respond to, together with results on the Italian and German parts of the corpus.
- 6) An ablation that changes only the source of the few-shot examples, which shows that examples drawn from old corpora, and not few-shot prompting itself, are what lower performance on AI-written phishing.
- 7) Operating-point analysis: how the ranking changes at a realistic amount of phishing, and what two-stage detectors built from the same models achieve at a fraction of the cost.

The whole study ran on one 8 GB laptop for under two US dollars of API fees. Code, raw model answers, results and a notebook are public.¹

II. Related Work

LLM detectors for phishing email. Koide et al. [2] prompt GPT-4 with a chain of thought and parse the verdict. Heiding et al. [1] use LLMs as judges of suspicious mail. Uddin et al. [3] fine-tune RoBERTa and add word level explanations. Lin et al. [4] study small LLMs of about three billion parameters, the same size range we use, and improve them with prompt engineering, LoRA fine-tuning on explanations from a paid teacher model, and ensembling; they list the missing cost analysis as a limitation. MultiPhishGuard [7] combines five agents and reports a 2.73% false positive rate. Phishsense-1B [8] is a phishing-specific 1B model that reports 97.5% on its own data and 70% on a separate real-world set. Kuikel et al. [5] find that the most accurate model is not the one with the most faithful explanations. A 2026 review [17] covers the field from both the attack and the defence side.

Cross-corpus evaluation. Two recent works already test across datasets, and we position ourselves against them rather than around them. E-PhishGen [9] reassesses eight legacy corpora with a leave-one-dataset-out protocol over classical models, DistilBERT and several LLMs, and releases E-PhishLLM, a corpus of emails written by GPT-4o and GPT-4o-mini, which we use as our AI-phishing test. Bhuiyan and Bhuiyan [10] study six corpora with leave-one-corpus-out training, prevalence shift and artifact learning for classical models. Fomin [15] reports the same kind of gap between pooled cross-validation and leave-one-dataset-out for malicious prompt classifiers, and Gutierrez et al. [16] show that much of the transfer gap to LLM-written phishing can be recovered by recalibrating the decision threshold. Neither E-PhishGen nor Bhuiyan and Bhuiyan measure or remove the overlap between the corpora they train and test on, and none of these works report what the detectors cost to run. Those two gaps,

together with false alarm rates on legitimate mail, are what this paper adds.

Leakage and deduplication. Arp et al. [12] list data snooping and sampling bias among the most common pitfalls of machine learning in security. Lee et al. [13] show that near-duplicates are pervasive in standard text corpora and that removing them is needed for honest evaluation; our matcher follows their MinHash approach [20]. Liu et al. [14] find near-identical samples leaking between train and test splits in Android malware detection, the closest analogue in security to what we find for email. For LLMs evaluated zero-shot there is the further worry that public corpora were part of pre-training, which a recent survey treats in detail [19].

Operating points. Al Barwani et al. [22] motivate their work by the false positive burden that standalone LLM classifiers create. Şahin and Mert [18] propose conformal deferral for security alert triage, which is the natural next step for the operating points we report.

III. Data

We use nine sources (Table I). Six contain both classes and take turns as the held-out corpus: SpamAssassin, CEAS-08, TREC-07, Ling-Spam and Enron from the curated collection of Champa et al. [11], and the Kaggle phishing email set used by [3]. Three are test only. Nazario contains real credential phishing and Nigerian contains advance fee fraud, both with no legitimate class. E-PhishLLM [9] contains phishing and legitimate emails written by GPT-4o and GPT-4o-mini; we use its English part as a test set and its Italian and German parts as a second test of language transfer.

For every source the model input is the subject and the body joined by a new line. Label 1 means phishing or spam and label 0 means legitimate. We drop bodies shorter than 20 characters and exact duplicates inside a corpus. Corpora larger than 10,000 emails are reduced to a stratified sample of 10,000 (seed 42) so that the three largest ones do not dominate training. From each source we also draw a fixed evaluation subset of 300 emails, balanced where both classes exist (150 for the one-class sets). Every model, local or paid, is scored on exactly these emails.

Two properties of this data matter for reading our results. The positive class is not the same across corpora: Section V-C measures what it actually contains. And the Kaggle set has no subject field, so its text is structurally unlike the other corpora, which is a difference that has nothing to do with phishing.

IV. Method

Fig. 1 shows the pipeline. Each step is one script and writes its own result files, so a run can be stopped and resumed.

¹<https://github.com/Jahid15/phishing-llm-cross-corpus>

TABLE I

Email sources after cleaning. “Used” is after the 10k stratified cap. “Eval” is the fixed subset every model is scored on.

Source	Role	Cleaned	Used	Positive	Eval
SpamAssassin	train/test	5,803	5,803	1,712	300
CEAS-08	train/test	39,091	10,000	5,574	300
TREC-07	train/test	53,638	10,000	5,460	300
Ling-Spam	train/test	2,858	2,858	458	300
Enron	train/test	29,604	10,000	4,689	300
Kaggle	train/test	17,505	10,000	3,731	300
Nazario	test	1,554	1,554	1,554	150
Nigerian	test	3,316	3,316	3,316	150
E-PhishLLM (en)	test	11,502	10,000	5,213	300
E-PhishLLM (it)	test	2,701	2,701	1,131	300
E-PhishLLM (de)	test	2,349	2,349	1,077	300

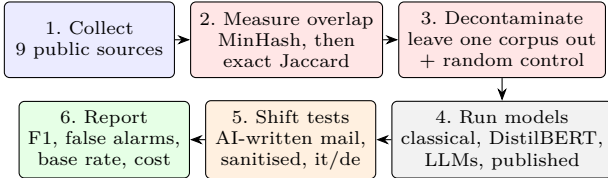


Fig. 1. The pipeline. Steps 2 and 3, and the operating-point part of step 6, are what earlier cross-corpus work does not do.

A. Measuring overlap between corpora

We compare every pair of the nine sources on the full cleaned corpora, before sampling, at three levels.

- Exact raw: MD5 of the text exactly as the model reads it.
- Exact normalized: MD5 of the body after lowercasing and removing all whitespace.
- Near duplicate: MinHash [20] with 128 permutations over 9-character shingles of the first 2,000 letters and digits of the body, with locality sensitive hashing to shortlist candidates at a deliberately loose threshold. Every shortlisted pair is then checked with the exact Jaccard similarity of the two shingle sets, and pairs at 0.8 or above count as the same email. The MinHash estimate alone has a standard error of about 0.035 at this threshold, which is too loose for a headline number, so it is used only to shortlist.

We first tried 5-word shingles. They missed many copies, because the Kaggle aggregate often deletes line breaks and glues two words together (“cream?Isn’t”). Character shingles over letters and digits ignore that.

Validating the matcher. Overlap is our main measurement, so it gets its own check. We take 400 random SpamAssassin emails that have no whitespace-exact match in Kaggle, compare each one against all 17,505 Kaggle emails by brute force, and compare that ground truth with what the pipeline decided for the same emails. We also report how the counts move when the Jaccard threshold is 0.7, 0.8 or 0.9.

Duplicates inside one corpus. The same matcher run within a corpus tells us how much of each evaluation subset

has a near copy in the part of its own corpus that a model would train on. This is what makes ordinary in-corpus scores optimistic even before any cross-corpus question is asked.

B. Leave one corpus out, with a control

For each of the six corpora T we train on the other five and test on T . We run this three ways.

- raw: the training data as it is.
- clean: every training email that has a near duplicate in T is removed first.
- random: the same number of training emails is removed, at random, with the class balance kept.

The third setting is the control that separates leakage from data volume. Decontamination also shrinks the training set, so on its own a drop from raw to clean could be explained by having less data. If the random setting stays close to raw while clean drops, the cause is the leaked emails.

The three test-only sets and the sanitised variants are scored with a model trained on all six corpora, again with near duplicates of the test set removed.

As a reference we report the in-corpus score two ways: the usual 80/20 stratified split inside each corpus, and, so that it can be compared with the other model families, a model trained on the corpus minus the shared evaluation subset and tested on that subset.

C. Models

Classical. TF-IDF over word unigrams and bigrams (50,000 features, minimum document frequency 2) with logistic regression or multinomial naive Bayes.

Fine-tuned encoder. distilbert-base-uncased [21], one epoch, 128 tokens, batch 16, learning rate 5×10^{-5} , on a stratified sample of 1,000 emails per training corpus (about 5,000 per fold), on the laptop CPU.

Small open LLMs. Llama-3.2-1B, Llama-3.2-3B, Llama-3.1-8B, Qwen-2.5-7B, Gemma-3-12B and Phi-4 (14B), instruction tuned, called through OpenRouter. The system prompt asks whether the email is “malicious or unwanted (phishing, scam or spam) or legitimate” and to answer with one word. Temperature is 0 and emails are cut to 1,500 characters. An LLM never sees our training data, so every test set is unseen for it.

Published phishing detectors. Three models that other people trained and released for this task: a BERT-large and a ModernBERT-large classifier, and an ungated merge of Phishsense-1B [8]. Their model cards name the Kaggle set, SpamAssassin, Enron, CEAS or Ling as training data, which is the same lineage we test on, so their scores on our legacy corpora are partly a memory test while E-PhishLLM is mail they cannot have seen. Phishsense is a causal model; its official prompt template sits in a gated repository we cannot read, so we prompt it exactly like our other LLMs and say so.

Commercial reference. Gemini-3.1-Flash-Lite and GPT-4o-mini under the same prompt, to show how far the cheap open models are from what a paid API gives today.

D. Reading the answers

An answer containing “legit” (or a negation such as “not phishing”) counts as negative, one containing “phish”, “spam” or “malicious” as positive. Some providers truncate the reply to a single token, so a prefix such as “ph” or “leg” is accepted as well. Anything else counts as no verdict, is treated as negative and is reported separately, because in a real pipeline an answer nobody can parse is a missed detection. Calls that failed after four retries are recorded with zero tokens and retried on the next run, and are counted apart from refusals. All raw answers are kept, so the parser can be improved and the answers re-read without paying for the calls again, which is what we did once in this study.

E. Metrics

We report F1 for the positive class, because accuracy hides the failure we care about, and next to it the false alarm rate, the share of legitimate emails that a model flags. The headline robustness number is the mean F1 over the six held-out corpora. Around it we give two kinds of uncertainty: a 1,000 sample bootstrap over emails, using the same resample positions for every model so that differences between models are paired, and a leave-one-corpus-out jackknife over the six corpora, which is the wider and more honest interval when the question is how a model behaves on a corpus nobody has seen. Paired differences between models are tested with an exact McNemar test.

Because the evaluation subsets are balanced and real mailboxes are not, we also report precision and F1 recomputed at lower phishing rates, keeping the measured recall and false alarm rate fixed.

Cost. For LLMs, cost is the sum of the charge OpenRouter reports for every call, over the same six corpora the F1 column uses, divided by the number of emails. It is the cost of emails truncated to 1,500 characters, so longer mail costs more. Local models have no API cost; we report their processing time instead. One caveat we cannot remove: OpenRouter routes a model to a provider of its choosing, so a small part of the cost difference between models is routing rather than the model itself.

F. Two more protocols

What the positives are. An LLM annotator sorts every positive email in the evaluation subsets into phishing or spam, with a second model annotating 300 of them so that agreement can be reported rather than assumed.

Explanation quality. On 100 emails two models are asked for a verdict and one sentence of reasoning, and a third model judges whether the reason points at something that is really in the email and whether it supports the

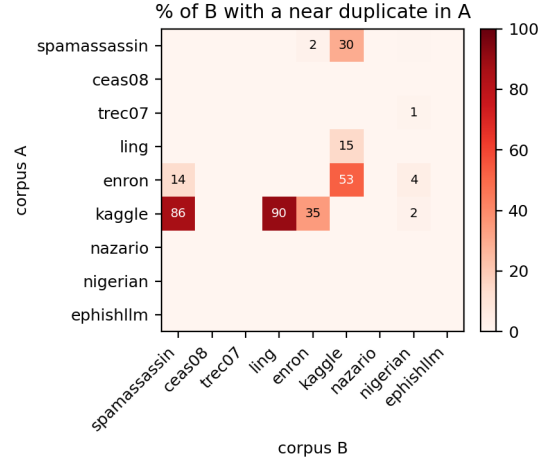


Fig. 2. Near-duplicate overlap. Cell (A, B) is the share of corpus B that has a near duplicate in corpus A .

verdict. We also read a sample by hand, because a model judging a model is not the last word.

V. Results

A. The corpora overlap, and exact matching does not see it

Table II and Fig. 2 show what the nine sources share. Compared exactly as a model reads them, SpamAssassin and Kaggle have 2 emails in common. With whitespace removed they have 3,850, and with near-duplicate matching 5,032, which is 87% of SpamAssassin. Kaggle also holds 91% of Ling-Spam and 36% of Enron, and 93% of the Kaggle corpus has a near copy in some other corpus. The Kaggle “dataset” is in large part three older corpora put together with the formatting changed on the way. CEAS-08, TREC-07, Nazario and E-PhishLLM share almost nothing with any other source, which gives us a control group for the next experiment.

The result does not depend on where we put the threshold (Table III): at a Jaccard of 0.9 the Kaggle share is still 86%. The matcher agrees with brute force on the hard cases: on 400 SpamAssassin emails with no whitespace-exact match in Kaggle, the pipeline has precision 1.00 and recall 1.00 against an exhaustive search, and 62% of those hard cases really do have a near copy in Kaggle. The choice of how much of each email to compare does not matter either: repeating the study over the first 5,000 characters instead of 2,000 moves the SpamAssassin figure from 86.7% to 87.1%.

Table III also shows that the problem starts inside each corpus. Between 10% and 29% of a corpus has a near duplicate elsewhere in the same corpus, and 9 to 19% of the evaluation emails have a near copy in the part of their own corpus that a model trains on. Ordinary in-corpus evaluation is therefore already optimistic before any cross-corpus question is asked.

TABLE II

Emails of one corpus found inside another, at three levels of matching: exact text, exact after removing all whitespace, and near duplicate (exact Jaccard at least 0.8). Percentages are of the first corpus.

Pair	Exact	No space	Near dup.
SpamAssassin in Kaggle	2 (<0.1%)	3,850 (66.3%)	5,032 (86.7%)
Ling-Spam in Kaggle	61 (2.1%)	62 (2.2%)	2,600 (91.0%)
Enron in Kaggle	3 (<0.1%)	187 (0.6%)	10,634 (35.9%)
Kaggle in Enron	3 (<0.1%)	188 (1.1%)	9,583 (54.7%)
SpamAssassin in Enron	0 (0.0%)	122 (2.1%)	855 (14.7%)
Nigerian in TREC-07	2 (<0.1%)	2 (<0.1%)	51 (1.5%)

TABLE III

Duplication inside and between corpora, in percent. “Internal” is the share of a corpus with a near copy elsewhere in the same corpus. “Eval in train” is the share of that corpus’s evaluation emails with a near copy in the part of the corpus a model trains on. The last three columns are the share with a near copy in a different corpus, at three Jaccard thresholds.

Corpus	Internal	Eval in train	≥ 0.7	≥ 0.8	≥ 0.9
SpamAssassin	13.7	17.3	88.6	87.2	85.4
CEAS-08	28.9	19.0	0.2	0.1	0.0
TREC-07	19.0	11.0	0.3	0.2	0.1
Ling-Spam	9.7	11.3	91.2	91.0	88.0
Enron	17.4	8.7	40.7	36.2	28.7
Kaggle	12.4	9.7	95.3	93.5	85.9
Nazario	21.9	22.0	0.0	0.0	0.0
Nigerian	36.1	33.3	10.4	6.1	3.0
E-PhishLLM (en)	0.0	0.0	0.0	0.0	0.0

B. Leaked emails inflate cross-corpus scores

Table IV repeats the single-corpus transfer test. Training on Kaggle and testing on SpamAssassin looks almost perfect at F1 0.966, and falls to 0.672 when the Kaggle training emails that also appear in SpamAssassin are removed. The pair that shares nothing, CEAS-08 to TREC-07, does not move at all.

Table V shows the same effect under leave-one-corpus-out, together with the control. Removing leaked emails costs logistic regression 12.1 points on Ling-Spam and 10.8 on SpamAssassin, while CEAS-08 and TREC-07, which share almost nothing with the others, move by 0.1 points or less. Removing the same number of training emails at random instead changes almost nothing: the random setting stays within 0.4 points of the raw setting on every corpus. So the drop is caused by which emails are removed, not by how many.

Fig. 3 puts the settings side by side. The usual in-corpus split gives 0.930 to 0.986. Honest leave-one-corpus-out gives 0.832 to 0.927. The in-corpus number overstates what the model will do on mail from a new source by 4 to 15 points.

C. What the benchmarks actually contain

Before comparing models it is worth knowing what the positive class is. Table VI gives the answer: across the six corpora only 15 to 25% of the positive evaluation

TABLE IV

Single-corpus transfer with TF-IDF and logistic regression, before and after removing training emails that also appear in the test corpus. The last row is a pair that shares almost nothing.

Train $\rightarrow$ test	Removed	Raw F1	Clean F1
Kaggle $\rightarrow$ SpamAssassin	2,954	0.966	0.672
Kaggle $\rightarrow$ Enron	5,511	0.969	0.661
Kaggle $\rightarrow$ Ling-Spam	1,450	0.979	0.837
SpamAssassin $\rightarrow$ Kaggle	5,032	0.766	0.590
SpamAssassin $\rightarrow$ Enron	855	0.673	0.440
CEAS-08 $\rightarrow$ TREC-07	6	0.737	0.734

TABLE V

Leave-one-corpus-out F1 of TF-IDF with logistic regression. “Removed” is the number of training emails that have a near duplicate in the held-out corpus. “Random” is the control that removes the same number of training emails at random. Δ is raw minus clean, in F1 points.

Held out	Removed	In-corp.	Raw	Random	Clean	Δ
SpamAssassin	3,198	0.966	0.947	0.947	0.840	+10.8
CEAS-08	17	0.986	0.833	0.833	0.832	+0.1
TREC-07	9	0.985	0.871	0.871	0.871	+0.0
Ling-Spam	1,460	0.930	0.962	0.960	0.841	+12.1
Enron	6,371	0.981	0.972	0.970	0.911	+6.1
Kaggle	11,206	0.965	0.976	0.972	0.927	+4.9

emails are phishing. The rest is bulk advertising. The two annotators agree on 86% of the 300 doubly annotated emails (Cohen’s κ 0.35, held down by the strong imbalance between the two classes).

This matters for how results in this field should be read. A model that scores well on these corpora is mostly proving that it can find spam. Every model we tested detects the phishing subset slightly better than the spam subset, by 1 to 14 points, so the ranking does not change, but the headline numbers describe a different task from the one the papers name.

D. Comparing model families on unseen mail

Table VII is the main comparison and Fig. 4 breaks it down per test set.

Small open LLMs are competitive on unseen corpora. Zero-shot Qwen-2.5-7B reaches a mean F1 of 0.928 over the six held-out corpora, above TF-IDF with logistic regression (0.899) and the fine-tuned DistilBERT (0.891), and it raises the fewest false alarms of the open models at 3% of legitimate mail. The paired bootstrap puts the difference to logistic regression at +0.029 (interval +0.016 to +0.044), and an exact McNemar test on the pooled 1,800 emails gives $p = 5e - 5$. The jackknife column is a reminder of how much one corpus can move the average.

Published detectors show the contamination from the other side. The released detectors behave as contamination predicts. BERT-phishing reaches 0.962 on our legacy corpora and 0.470 on AI-written phishing; Phishsense-1B reaches 0.947 on our legacy corpora and 0.839 on AI-written phishing; ModernBERT-phishing reaches 0.892

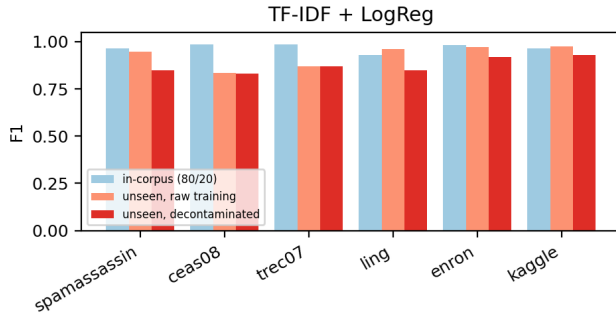


Fig. 3. TF-IDF with logistic regression on each held-out corpus: in-corpora split, unseen corpus with raw training data, and unseen corpus after decontamination.

TABLE VI

What the positive class contains, according to an LLM annotator (second annotator on 300 emails, agreement 86%). Percentages of the positive evaluation emails.

Corpus	Positives	Phishing (%)	Spam (%)
SpamAssassin	150	18.7	81.3
CEAS-08	150	14.7	85.3
TREC-07	150	24.7	75.3
Ling-Spam	150	19.3	79.3
Enron	150	21.3	78.7
Kaggle	150	25.3	74.7

on our legacy corpora and 0.149 on AI-written phishing. BERT-phishing has the highest unseen-corpus score in the whole table (0.962), above every LLM we ran, which is what a leaderboard built on these corpora would report. Its model card names the same Kaggle lineage we test on, so that column is partly a memory test, and the AI-written column is where the difference shows. The one exception is the LoRA-tuned causal model, which transfers much better than the two encoder classifiers, so the failure is not a property of released models as such. We also checked that this is not simply the decision threshold. The ModernBERT card recommends 0.37 rather than the default. On AI-written phishing its recall goes from 0.09 at the default to 0.11 at the recommended threshold and 0.22 at 0.2, while the false alarm rate on the same set moves from 8% to 33%. Lowering the threshold trades one failure for the other rather than removing it.

A current commercial model is still ahead. Gemini-3.1-Flash-Lite reaches 0.960 on unseen corpora with 1.7% false alarms and 0.950 on AI-written phishing, better than every open model we ran on all three axes, at 0.080 dollars per 1,000 emails. The honest summary for a small organisation is not that open models win, but that they cost less and lose some accuracy, and the table says how much of each.

The best in-corpora model is among the worst on new attacks. DistilBERT has the highest in-corpora F1 in the table (0.975) and one of the lowest scores on AI-written phishing (0.540). Ranking by the usual measure would pick it first. It is also the most truncated model in our

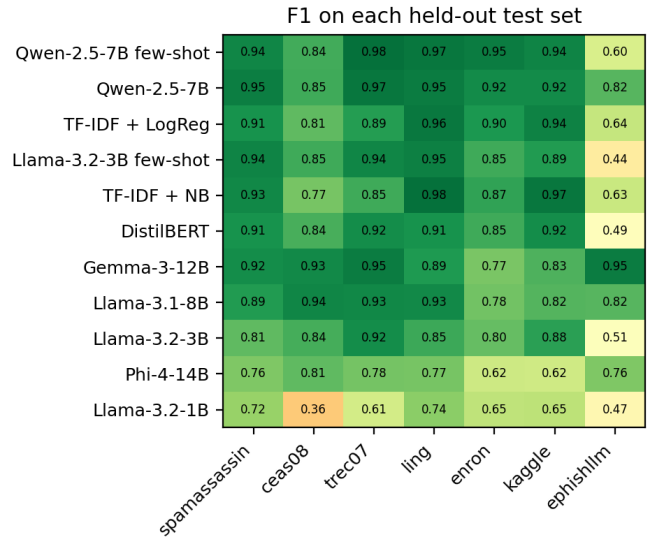


Fig. 4. F1 of every model on each held-out test set.

setup at 128 word pieces, which is a second reason to read that row carefully.

Format compliance is part of capability. Phi-4 ignored the one-word instruction on 37% of emails, starting an explanation that the token limit then cut off, and those answers count as missed detections. Given room to answer it behaves quite differently (unseen-corpus F1 0.754 against 0.727, with 4% unparsed instead of 37%), which is a warning about how much of a published LLM score can depend on output formatting rather than detection.

E. AI-written phishing, and a token that looked like a shortcut

E-PhishGen writes phishing links as the literal placeholder «link». In our English evaluation subset 94 of the 150 phishing emails contain such a placeholder and 0 of the 150 legitimate ones do. A detector could react to the token instead of to the message, which would make our AI-phishing numbers meaningless, so we tested it two ways.

The first test is the one that comes to mind: split the phishing emails by whether they carry a placeholder and compare recall. This gives a large gap, 16 points on average, and it is misleading. The two halves are different emails: the ones with a placeholder are link-based lures, the ones without are business email compromise and similar text-only attacks, which are harder for any detector.

The second test is paired and is the one that answers the question. We rebuild the same 300 emails with the placeholder removed, and with it replaced by a plausible URL, and compare each model’s verdict on the same email before and after. The mean change in recall is 0.9 points, and the largest for any model is 9.6 points. Table VIII shows the F1 on all three versions: they differ by little,

TABLE VII

Main results, all on the same evaluation emails. Trained models use decontaminated leave-one-corpus-out data. “Unseen” is the mean F1 over the six held-out corpora, with a paired bootstrap interval over emails and a leave-one-corpus-out jackknife range over corpora. FA is the share of legitimate emails flagged. AI phishing is F1 on E-PhishLLM. Nazario contains only phishing, so we report recall. Cost is the measured charge in USD per 1,000 emails on the six corpora.

Model	In-corpus	Unseen	Bootstrap	Jackknife	FA (%)	AI phish	Nazario	USD/1k
TF-IDF + LogReg	0.968	0.899	0.882–0.914	0.886–0.917	9.6	0.640	0.713	local
TF-IDF + NB	0.884	0.894	0.879–0.909	0.877–0.918	6.6	0.631	0.600	local
DistilBERT	0.975	0.891	0.876–0.906	0.885–0.902	6.3	0.540	0.780	local
BERT-phishing (published)	–	0.962	0.952–0.971	0.955–0.982	2.8	0.470	0.893	local
Phishsense-1B (published)	–	0.947	0.935–0.958	0.938–0.967	7.0	0.839	1.000	local
ModernBERT-phishing (published)	–	0.892	0.876–0.906	0.882–0.901	3.4	0.149	0.513	local
Qwen-2.5-7B few-shot mixed	–	0.948	0.936–0.958	0.944–0.953	4.9	0.673	–	0.083
Qwen-2.5-7B few-shot	–	0.937	0.924–0.949	0.928–0.955	4.8	0.598	0.847	0.075
Qwen-2.5-7B few-shot AI	–	0.930	0.916–0.944	0.920–0.940	10.4	0.723	–	0.079
Qwen-2.5-7B	–	0.928	0.915–0.941	0.921–0.944	3.0	0.819	0.940	0.033
Llama-3.2-3B few-shot AI	–	0.903	0.886–0.919	0.889–0.919	9.9	0.649	–	0.040
Llama-3.2-3B few-shot	–	0.901	0.885–0.917	0.892–0.912	2.7	0.438	0.680	0.038
Gemma-3-12B	–	0.882	0.859–0.901	0.867–0.905	25.1	0.955	0.987	0.017
Llama-3.1-8B	–	0.881	0.859–0.902	0.870–0.901	21.4	0.817	0.973	0.012
Llama-3.2-3B	–	0.850	0.831–0.869	0.836–0.861	5.1	0.507	0.827	0.017
Phi-4-14B relaxed	–	0.754	0.716–0.788	0.746–0.762	48.0	0.700	0.773	0.029
Phi-4-14B	–	0.727	0.694–0.753	0.710–0.748	27.6	0.759	0.660	0.022
Llama-3.2-1B	–	0.623	0.580–0.662	0.600–0.676	59.9	0.474	0.760	0.009
Gemini-3.1-Flash-Lite	–	0.960	0.950–0.968	0.955–0.973	1.7	0.950	0.987	0.080
GPT-4o-mini	–	0.950	0.939–0.961	0.944–0.961	4.3	0.917	0.987	0.046

and for several models the sanitised version scores slightly higher.

So the token is not what the detectors were keying on, and the AI-phishing results stand. We report it because the naive test would have supported the opposite conclusion, and because anyone using this corpus should know the token is there. The same caution applies to the legitimate half of the corpus, which is synthetic as well: a detector that flags synthetic business mail is being tested on two shifts at once.

On the Italian and German parts, prompted in English, the better models do not lose much (Gemini 0.966, Gemma 0.948 on Italian) while the smaller ones move more, so language transfer depends on the model rather than being a uniform penalty.

F. Where few-shot examples come from matters

Four examples drawn from the other legacy corpora raise Qwen-2.5-7B from 0.928 to 0.937 on the held-out corpora, which the paired bootstrap cannot separate from zero (difference +0.009, interval -0.001 to +0.018), and lower its AI-phishing F1 from 0.819 to 0.598, which is significant (McNemar $p = 2e - 12$). Replacing those examples with AI-written ones, and changing nothing else, recovers the AI-phishing score to 0.723 at the price of a higher false alarm rate (10% against 3%). A mixed prompt lands between the two (0.673). The same pattern holds for Llama-3.2-3B.

The lesson is not that few-shot prompting is bad. It is that the examples act as a description of what phishing

looks like, so examples from 2002 to 2008 teach a model to expect mail from that era.

G. Operating points: base rate and two-stage detectors

Our evaluation sets are balanced, real mailboxes are not. Table IX recomputes precision at a 5% phishing rate, keeping the measured recall and false alarm rate. The ranking changes: Gemma-3-12B, which is the best open model on AI-written phishing, drops to F1 0.289 because it would flag 239 legitimate emails in every thousand, while Qwen stays at 0.725 with 28.

Table X builds two-stage detectors from models we have already measured, with no new calls. Sending only the mail that TF-IDF flags to Qwen-2.5-7B gives F1 0.911 with 1.1% false alarms at 0.0161 dollars per 1,000 emails, which is half the cost of running Qwen on everything and a lower false alarm rate than either model alone. In the other direction, letting Gemma see the mail that Qwen cleared raises recall on AI-written phishing to 0.987 at the price of 25% false alarms, which is the shape of a quarantine queue rather than a blocker.

H. Explanations

On 100 emails, both small models always produced a reason when asked. A judge model found the reason grounded in the email for 91% of Qwen’s answers and 92% of Gemma’s. The interesting part is what happens when the verdict is wrong: then only 53% of the reasons are grounded, that is, a wrong verdict tends to come with an invented justification. Reading a sample by hand agrees with the judge. This supports the concern of Kuikel et

TABLE VIII

AI-written phishing. The first column is the corpus as published. The next two are the same 300 emails with the give-away placeholder token removed or replaced by a plausible URL. Then the Italian and German parts. The last column is the recall gap on the English set between phishing emails that contain the token and those that do not.

Model	As published	Token removed	Token replaced	Italian	German	Token gap
TF-IDF + LogReg	0.640	0.635	0.635	0.667	0.673	+0.16
DistilBERT	0.540	0.540	0.558	0.662	0.639	+0.09
Qwen-2.5-7B	0.819	0.805	0.876	0.762	0.842	+0.22
Gemma-3-12B	0.955	0.949	0.952	0.948	0.927	+0.04
Llama-3.1-8B	0.817	0.831	0.894	0.924	0.932	+0.29
Gemini-3.1-Flash-Lite	0.950	0.943	0.950	0.966	0.948	+0.14

TABLE IX

The same models at a realistic amount of phishing. Recall and false alarm rate are measured; precision and F1 are recomputed for a mailbox where 5% of mail is phishing. The last column is how many legitimate emails out of 1,000 would be flagged.

Model	Recall	FA (%)	Prec. at 50%	Prec. at 5%	F1 at 5%	Bad alerts/1k
Gemini-3.1-Flash-Lite	0.940	1.7	0.983	0.748	0.833	16
Qwen-2.5-7B	0.893	3.0	0.968	0.610	0.725	28
DistilBERT	0.857	6.3	0.931	0.416	0.560	60
TF-IDF + LogReg	0.898	9.6	0.904	0.331	0.484	91
Llama-3.1-8B	0.948	21.4	0.815	0.189	0.315	204
Gemma-3-12B	0.977	25.1	0.795	0.170	0.289	239
Phi-4-14B	0.732	27.6	0.727	0.123	0.210	262

TABLE X

Two-stage detectors built from models we already measured, with no new calls. “confirm” sends only the mail stage 1 flagged to stage 2 and needs both to agree; “rescue” sends only the mail stage 1 cleared and accepts either verdict. “To stage 2” is the share of mail that reaches the second model, which is what makes the cost lower than running that model on everything.

Pipeline	Policy	Unseen F1	FA (%)	AI phish	To stage 2 (%)	USD/1k
TF-IDF + LogReg → Gemma-3-12B	confirm	0.914	4.7	0.750	50	0.0083
TF-IDF + LogReg → Gemma-3-12B	rescue	0.869	30.0	0.836	50	0.0084
TF-IDF + LogReg → Qwen-2.5-7B	confirm	0.911	1.1	0.678	50	0.0161
TF-IDF + LogReg → Qwen-2.5-7B	rescue	0.916	11.4	0.753	50	0.0164
Llama-3.1-8B → Qwen-2.5-7B	rescue	0.894	21.8	0.870	42	0.0256
Gemma-3-12B → Qwen-2.5-7B	rescue	0.882	25.4	0.955	39	0.0292
Llama-3.1-8B → Qwen-2.5-7B	confirm	0.914	2.7	0.760	58	0.0309
Gemma-3-12B → Qwen-2.5-7B	confirm	0.929	2.7	0.819	61	0.0366
Qwen-2.5-7B → Gemma-3-12B	confirm	0.929	2.7	0.819	46	0.0402
Qwen-2.5-7B → Gemma-3-12B	rescue	0.882	25.4	0.955	54	0.0415

al. [5] that accuracy and explanation quality are different things, and it argues against showing a model’s reason to an end user without checking the verdict first.

I. Cost

Zero-shot classification cost between 0.009 and 0.033 US dollars per 1,000 emails for the open models and 0.080 for Gemini-3.1-Flash-Lite, measured from the charge reported for every call on 1,500-character emails. TF-IDF needs 0.09 to 0.40 CPU seconds per 1,000 emails for inference and no API. DistilBERT needs 7 minutes of laptop CPU time per training fold and 22 seconds per 1,000 emails. The whole study, 40,498 model calls including the ones we discarded, cost 1.58 US dollars.

VI. Discussion

Measure overlap before claiming generalization. The clearest finding here is about the benchmarks rather than the models. A paper that trains on the Kaggle aggregate

and tests on SpamAssassin is testing mostly on its own training data, and exact-match deduplication does not reveal it because the aggregate changed the line breaks. A near-duplicate pass over 170,000 emails took five minutes on a laptop in our run, so there is no practical reason to skip it before a cross-dataset claim. Our random-removal control also shows that the effect is leakage and not the smaller training set, which is the first question a careful reader asks.

In-corpus scores answer a different question. The 97 to 99% in the literature is real, but it says how well a model recognises more mail from the same collection. A mail server does not receive mail from the training collection. On unseen corpora the same model loses several points; on AI-written mail it loses many more; and the model with the best in-corpus score in our table is among the weakest on AI-written phishing.

Know what the positive class is. Only a fifth of the

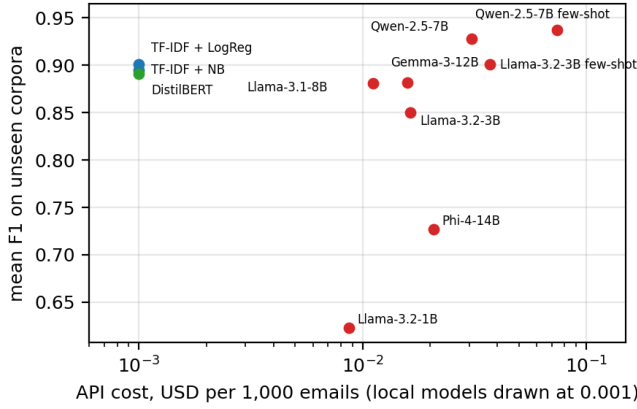


Fig. 5. Mean F1 on unseen corpora against measured API cost. Local models have no API cost and are drawn at 0.001.

positive emails in these corpora are phishing. Work that reports “phishing detection” on them is largely reporting spam detection. This does not invalidate the corpora, but a paper should either say which it means or, as we did, measure the split.

Choose an operating point, not a leaderboard position. At the balanced prevalence used in benchmarks, several models look similar. At a realistic phishing rate the false alarm rate dominates, and models that share an F1 differ by an order of magnitude in how much legitimate mail they would flag. The two-stage detectors show a more practical way to use a cheap model: a free classical filter in front of an LLM cuts both the cost and the false alarms, and the reverse order is the right shape for a quarantine queue that a person reviews. Conformal deferral [18] is the natural next step.

What we would deploy. For an organisation with no GPU and a small budget, a TF-IDF filter in front of zero-shot Qwen-2.5-7B is the best combination we measured: it is the cheapest configuration in our tables and has the lowest false alarm rate. If the budget allows a commercial API, Gemini-3.1-Flash-Lite was better than every open model on every axis we measured, which is an honest result even though it is not the one we hoped for. Gemma-3-12B should be used only where a person reviews the output.

Test the shortcut you suspect, on the same emails. The AI-written corpus writes links as a placeholder that appears almost only in phishing mail, and the obvious comparison, splitting the phishing emails by whether they carry it, says the token is worth a large amount of recall. That comparison is confounded, because the two halves are different kinds of attack. Removing the token from the same emails moves the verdicts by almost nothing. We report both because the quick test pointed the wrong way, and a reader who only ran it would have drawn the opposite conclusion about this corpus.

VII. Limitations

- **Labels.** Four corpora count ordinary spam as positive. We measure the split with an LLM annotator and a second annotator for agreement, but the underlying labels stay as published, and the annotator is itself a model.
- **Sample size.** Each test set has 300 emails, 150 for the one-class sets. The bootstrap interval on the mean F1 is about ± 0.015 ; the leave-one-corpus-out jackknife is wider and is the number to read when asking about a new corpus.
- **Input budgets differ by model family.** TF-IDF sees 20,000 characters, the LLMs 1,500, DistilBERT 128 word pieces. DistilBERT is therefore the most truncated model and its row should be read as a lower bound.
- **Training budgets differ.** DistilBERT trains on about 5,000 emails per fold against 40,000 for the classical models, to fit the laptop.
- **LLM pre-training.** The legacy corpora may be inside the LLMs’ pre-training data [19]. E-PhishLLM was released in 2025, after most of these models, which makes it the most trustworthy of our tests, and the published detectors we ran are contaminated by construction.
- **Providers.** OpenRouter routes each model to a provider of its choosing, so a small part of the cost and a small part of the behaviour is routing rather than the model.
- **Scope.** Text only, one prompt per model, one run. Headers, URLs and attachments are ignored, and the Kaggle corpus has no subject line at all, which is an unmeasured difference between it and the others.
- **Phishsense-1B** is evaluated through an ungated third-party merge and with our own prompt, because the original repository and its prompt template are gated. Its numbers may understate the released model.

VIII. Conclusion

Public phishing corpora overlap heavily, and in a way that exact matching cannot see: most of the Kaggle aggregate has a near duplicate in another public corpus. Removing those duplicates lowers cross-corpus F1 by up to 30 points for a single pair, while removing the same number of emails at random changes nothing. On decontaminated, unseen data a small open LLM used zero-shot is as good as a trained classical model and much better on AI-written phishing, for a few cents per thousand emails, though a current commercial model is still ahead of all of them. Which model is best depends on the operating point: at a realistic phishing rate the false alarm rate decides, and a free classical filter in front of a small LLM beats either part alone on both cost and false alarms. Finally, the corpora themselves deserve more scepticism than they usually get: four fifths of their positive emails are spam rather than phishing, and a corpus feature that looks

like a shortcut needs a paired test before it is believed, in either direction. High accuracy on one dataset is easy. Honest evaluation is the hard part. All code, raw model answers and results are public so that others can check and extend this work.

Acknowledgment

We thank our Computer Security course instructor at United International University for the feedback on our literature review and proposal.

References

- [1] F. Heiding, B. Schneier, A. Vishwanath, J. Bernstein, and P. S. Park, "Devising and detecting phishing emails using large language models," *IEEE Access*, vol. 12, pp. 42131–42146, 2024.
- [2] T. Koide, N. Fukushi, H. Nakano, and D. Chiba, "ChatSpamDetector: Leveraging large language models for effective phishing email detection," *arXiv:2402.18093*, 2024.
- [3] M. A. Uddin, M. Mahiuddin, and I. H. Sarker, "An explainable transformer-based model for phishing email detection: A large language model approach," *arXiv:2402.13871*, 2024.
- [4] Z. Lin, Z. Liu, and H. Fan, "Improving phishing email detection performance of small large language models," *arXiv:2505.00034*, 2025.
- [5] S. Kuikel, A. Piplai, and P. Aggarwal, "Evaluating large language models for phishing detection, self-consistency, faithfulness, and explainability," *arXiv:2506.13746*, 2025.
- [6] A. Alhuzali, A. Alloqmani, M. Aljabri, and F. Alharbi, "In-depth analysis of phishing email detection: Evaluating the performance of machine learning and deep learning models across multiple datasets," *Applied Sciences*, vol. 15, no. 6, 2025.
- [7] Y. Xue, E. Spero, M. W. Woo, W. Gao, and G. Russello, "MultiPhishGuard: An explainable and adaptive multi-agent LLM system for phishing email detection," *arXiv:2505.23803*, 2025.
- [8] S. E. Blake, "Phishsense-1B: A technical perspective on an AI-powered phishing detection model," *arXiv:2503.10944*, 2025.
- [9] L. Pajola, E. Caripoti, S. Banzer, S. Pizzi, M. Conti, and G. Apruzzese, "E-PhishGen: Unlocking novel research in phishing email detection," in *Proc. ACM Workshop on Artificial Intelligence and Security (AISec)*, 2025, pp. 64–76.
- [10] M. Bhuiyan and M. Bhuiyan, "The fragility of phishing detection models: Evidence from cross-corpus transfer, prevalence shift, artifact learning, and evasion risk," *Big Data and Cognitive Computing*, vol. 10, no. 7, art. 211, 2026.
- [11] A. I. Champa, M. F. Rabbi, and M. F. Zibran, "Curated datasets and feature analysis for phishing email detection with machine learning," in *Proc. IEEE 3rd Int. Conf. Computing and Machine Intelligence (ICMI)*, 2024, pp. 1–7.
- [12] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. USENIX Security Symposium*, 2022, pp. 3971–3988.
- [13] K. Lee, D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch, and N. Carlini, "Deduplicating training data makes language models better," in *Proc. ACL*, 2022, pp. 8424–8445.
- [14] G. Liu, C. Caragea, X. Ou, and S. Roy, "The impact of train-test leakage on machine learning-based Android malware detection," *arXiv:2410.19364*, 2025.
- [15] M. Fomin, "When benchmarks lie: Evaluating malicious prompt classifiers under true distribution shift," *arXiv:2602.14161*, 2026.
- [16] D. Gutierrez, W. Villegas-Ch, and J. Govea, "Cross-model evaluation of phishing detectors against LLM-generated emails," *Frontiers in Big Data*, vol. 9, 2026.
- [17] K. Sivaneswaran, C. Hewage, S. Herath, H. Rathore, R. Singh, and F. Jiang, "A systematic literature review of large language models in phishing attack generation and detection," *Array*, vol. 30, art. 100775, 2026.
- [18] E. Şahin and M. Mert, "Dual-threshold conformal deferral for trustworthy security alert triage," *Electronics*, vol. 15, no. 18, art. 4084, 2026.
- [19] M. Ravaut, B. Ding, F. Jiao, H. Chen, X. Li, R. Zhao, C. Qin, C. Xiong, and S. Joty, "A comprehensive survey of contamination detection methods in large language models," *Transactions on Machine Learning Research*, 2025.
- [20] A. Z. Broder, "On the resemblance and containment of documents," in *Proc. Compression and Complexity of Sequences*, 1997, pp. 21–29.
- [21] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv:1910.01108*, 2019.
- [22] S. Al Barwani, A. Amara Korba, and A. Anwar, "User-centric phishing detection: A RAG and LLM-based approach," *arXiv:2601.21261*, 2026.